ジオメトリック・インテリジェンス ケーススタディ

補助文書 03

Python のコード

—— 敵の多様体 ——

もし中国が同じ技術を持っていたら

Étale Cohomology

2026 年 11 月　License: CC BY 4.0

本書は、ケーススタディ・ノベル「敵の多様体」（以下、本編）に登場する数学を、小さな数値例で確かめるための Python のコード 13 本の全文と、その実行結果をまとめた文書である。本編には、これまでの作品のように図を描くためのコードはない。その代わり、補助文書 02 で解説した数学の性質 —— 検閲が多様体の品質指標をどう変えるか、均衡の計算がいつ収束するか、情報量がなぜ増えないか、因果の向きをどう測るか、など —— を、読者が自分の手で確かめられるようにした。

各コードは、本物の天網や Eight Eyes の多様体を再現するものではない。考え方を見るための、最小限の例である。数値はすべて架空のものである。

目次

- 第 1 章　実行方法
- 第 2 章　ランダム化回答法（tk01_randomized_response.py）
  - 実行結果
  - コードの全文
- 第 3 章　ベンフォードの法則（tk02_benford.py）
  - 実行結果
  - コードの全文
- 第 4 章　正則化が曲率にもたらす偏り（tk03_regularization_bias.py）
  - 実行結果

  - コードの全文
- 第 5 章　ランダース計量 —— 行きと帰りで長さの異なる定規（tk04_randers_asymmetry.py）
  - 実行結果
  - コードの全文
- 第 6 章　政策の順序効果 —— リー括弧と三重のリー括弧（tk05_bch_ordering.py）
  - 実行結果
  - コードの全文
- 第 7 章　命題 1.1 の条件 2 と条件 3 —— 検閲が近傍保存率と最小特異値に与える影響（tk06_manifold_conditions.py）
  - 実行結果
  - コードの全文
- 第 8 章　反復ベストレスポンスの収束と、ヘッセ行列による安定性の判定（tk07_iterated_best_response.py）
  - 実行結果
  - コードの全文
- 第 9 章　相互情報量とデータ処理不等式、KL 情報量とワッサースタイン距離（tk08_information.py）
  - 実行結果
  - コードの全文
- 第 10 章　移転エントロピーと偶然の一致の検定（tk09_causal_tests.py）
  - 実行結果
  - コードの全文
- 第 11 章　シンプレクティック積分 —— 長期の計算でエネルギーが崩れないこと（tk10_symplectic.py）
  - 実行結果
  - コードの全文
- 第 12 章　パーシステント・ホモロジーの β0 —— 2 つのかたまりがつながる（tk11_beta0_bridge.py）
  - 実行結果
  - コードの全文
- 第 13 章　ベイズ説得 —— 嘘をつかずに判断を動かす（tk12_bayesian_persuasion.py）

    - 実行結果
    - コードの全文
- 第 14 章　N 人囚人のジレンマとメカニズムデザイン（tk13_n_person_dilemma.py）
    - 実行結果
    - コードの全文

第 1 章　実行方法

```
pip install -r code/requirements.txt
python code/tk01_randomized_response.py
```

必要なライブラリは numpy と scipy である。乱数を使うコードは乱数の種を固定しているため、何度実行しても同じ結果が得られる。執筆者は、Python 3.12.3、numpy 2.5.4、scipy 1.18.1 の環境で、13 本すべてが正しく動作することを確認した（2026 年 11 月）。

コード	内容	本編の場面	解説
tk01_randomized_response.py	ランダム化回答法	第 9 章 9.5	補助文書 02 第 13 章 13.3
tk02_benford.py	ベンフォードの法則	第 8 章 8.4	補助文書 02 第 14 章 14.6
tk03_regularization_bias.py	正則化が曲率にもたらす偏り	第 2 章 2.7、第 3 章 3.5	補助文書 02 第 4 章 4.3
tk04_randers_asymmetry.py	ランダース計量 —— 行きと帰りで長さの異なる定規	第 2 章 2.7	補助文書 02 第 6 章
tk05_bch_ordering.py	政策の順序効果 —— リー括弧と三重のリー括弧	第 2 章 2.7	補助文書 02 第 7 章 7.3
tk06_manifold_conditions.py	命題 1.1 の条件 2 と条件 3 —— 検閲が近傍保存率と最小特異値に与える影響	第 2 章 2.6	補助文書 02 第 4 章 4.1・4.2

コード	内容	本編の場面	解説
tk07_iterated_best_response.py	反復ベストレスポンスの収束と、ヘッセ行列による安定性の判定	第６章 6.2・6.3	補助文書 02 第 10 章 10.4・10.5
tk08_information.py	相互情報量とデータ処理不等式、KL 情報量とワッサースタイン距離	第３章 3.4、第１０章 10.2	補助文書 02 第 15 章
tk09_causal_tests.py	移転エントロピーと偶然の一致の検定	第５章 5.3	補助文書 02 第 14 章 14.3・14.4
tk10_symplectic.py	シンプレクティック積分 —— 長期の計算でエネルギーが崩れないこと	第１２章	補助文書 02 第 16 章
tk11_beta0_bridge.py	パーシステント・ホモロジーの β0 —— ２つのかたまりがつながる	第５章 5.3	補助文書 02 第 8 章 8.1
tk12_bayesian_persuasion.py	ベイズ説得 —— 嘘をつかずに判断を動かす	第９章 9.6	補助文書 02 第 13 章 13.4
tk13_n_person_dilemma.py	N 人囚人のジレンマとメカニズムデザイン	第９章 9.1・9.2	補助文書 02 第 13 章 13.1・13.2

第２章　ランダム化回答法（tk01_randomized_response.py）

本編の場面：第９章 9.5　**解説**：補助文書 02 第 13 章 13.3

実行結果

```
本当の割合 0.300　推定 0.287　95%区間 [0.252, 0.322]
「はい」と答えた人が本当に「はい」である確率 0.462（１人の答えからは断定できない）
直接に尋ねた場合と比べた標準誤差の倍率 2.09（秘密を守る代わりに、推定の精度は下がる）
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 1：ランダム化回答法（Warner 型の一種、強制回答方式）
2,850 人の地方の幹部が、「公式の統計に載っていない暴動や債務があるか」に答える。
硬貨の表（確率 p）なら正直に答え、裏なら必ず「はい」と答える。
「はい」の割合 λ から、本当の割合 π を π_hat = (λ − (1 − p)) / p で推定する。数値は架空のもの。
"""
import numpy as np
rng = np.random.default_rng(2033)
N, pi_true, p = 2850, 0.30, 0.5
truth = rng.random(N) < pi_true        # 本当の答え（集める側には見えない）
heads = rng.random(N) < p              # 硬貨（集める側には見えない）
answer = np.where(heads, truth, True)     # 表なら正直、裏なら「はい」
lam = answer.mean()
pi_hat = (lam - (1 - p)) / p
se = np.sqrt(lam * (1 - lam) / N) / p     # 推定の標準誤差
# 「はい」と答えた人が、本当に「はい」である確率（１人１人の秘密がどれだけ守られるか）
post = (pi_true * 1) / (pi_true + (1 - pi_true) * (1 - p))
print(f"本当の割合 {pi_true:.3f}  推定 {pi_hat:.3f}  95%区間 [{pi_hat-1.96*se:.3f}, {pi_hat+1.96*se:.3f}]")
print(f"「はい」と答えた人が本当に「はい」である確率 {post:.3f}（１人の答えからは断定できない）")
print(f"直接に尋ねた場合と比べた標準誤差の倍率 {se/np.sqrt(pi_true*(1-pi_true)/N):.2f}（秘密を守る代わりに、推定の精度は下がる）")
```

第 3 章　ベンフォードの法則（tk02_benford.py）

本編の場面：第 8 章 8.4　**解説**：補助文書 02 第 14 章 14.6

実行結果

```
自然に生まれた数値：先頭が 1 の割合 0.319（ベンフォードでは 0.301）　カイ二乗 8.2　p 値 0.41
```

人の手で作られた数値：先頭が 1 の割合 0.000（ベンフォードでは 0.301）　カイ二乗 10984.9　p 値 0
注意：範囲の狭い数値や目標値の決まった数値には、もともとベンフォードの法則は当てはまらない。

コードの全文

```
"""
敵の多様体 補助文書 03 コード 2：ベンフォードの法則
自然に生まれる数値（桁が大きく違うものが混ざった数値）と、人の手で作られた数値の、先頭の桁の分布を比べる。
数値は架空のもの。外れていることは、不正の証拠ではなく、調べる場所の手がかりにすぎない。
"""
import numpy as np
from scipy import stats
rng = np.random.default_rng(7)
benford = np.log10(1 + 1 / np.arange(1, 10))
def first_digit(x): return (x / 10 ** np.floor(np.log10(x))).astype(int)
natural = rng.lognormal(mean=10, sigma=2.5, size=3000)       # 何桁にもわたって散らばる数値
made = rng.uniform(4500, 6500, size=3000)                    # 目標の近くに作られた数値（範囲が狭い）
for name, x in [("自然に生まれた数値", natural), ("人の手で作られた数値", made)]:
    obs = np.bincount(first_digit(x), minlength=10)[1:]
    chi2, pval = stats.chisquare(obs, benford * obs.sum())
    print(f"{name}: 先頭が 1 の割合 {obs[0]/obs.sum():.3f}（ベンフォードでは {benford[0]:.3f}）　カイ二乗 {chi2:.1f}　p 値 {pval:.2g}")
print("注意：範囲の狭い数値や目標値の決まった数値には、もともとベンフォードの法則は当てはまらない。")
```

第 4 章　正則化が曲率にもたらす偏り（tk03_regularization_bias.py）

本編の場面：第 2 章 2.7、第 3 章 3.5　**解説**：補助文書 02 第 4 章 4.3

実行結果

```
u（極からの角度）: [0.05 0.1  0.3  0.6  1.   1.5 ]
ε=0.0      曲率 K: [1. 1. 1. 1. 1. 1.]
ε=0.0001   曲率 K: [-1.3818e+01  1.3000e-02  9.8700e-01  9.9900e-01  1.0000e+00  1.000
```

```
0e+00]
ε=0.01     曲率 K: [-63.662 -24.334  -0.066   0.907   0.98    0.99 ]
```
正しい曲率はどこでも 1。ε を足すと、計量がつぶれかけていた極の近くほど、曲率が大きく偏る。

コードの全文

```
"""
敵の多様体 補助文書 03 コード 3：正則化が曲率に入れる偏り
球面の計量 ds² = du² + sin²u dv²（曲率はどこでも 1）は、u → 0 で sin²u → 0 となり、計量がつぶれかける。
計量に小さな正の値 ε を足して ds² = du² + (sin²u + ε) dv² とすると、数値は安定するが、曲率に偏りが入る。
F=0 の 2 次元の計量 E=1, G=G(u) のガウス曲率は K = −(√G)''/√G で計算できる。
"""
import numpy as np
u = np.array([0.05, 0.1, 0.3, 0.6, 1.0, 1.5])
h = 1e-4
def K(u, eps):
    g = lambda x: np.sqrt(np.sin(x) ** 2 + eps)
    d2 = (g(u + h) - 2 * g(u) + g(u - h)) / h ** 2
    return -d2 / g(u)
print("u（極からの角度）:", u)
for eps in [0.0, 1e-4, 1e-2]:
    print(f"ε={eps:<7} 曲率 K:", np.round(K(u, eps), 3))
print("正しい曲率はどこでも 1。ε を足すと、計量がつぶれかけていた極の近くほど、曲率が大きく偏る。")
```

第 5 章　ランダース計量 ―― 行きと帰りで長さの異なる定規（tk04_randers_asymmetry.py）

本編の場面：第 2 章 2.7　**解説**：補助文書 02 第 6 章

実行結果

```
b=+0.0: 緊張を高める（A→B）1.00　緊張を下げる（B→A）1.00　比 1.00
b=-0.3: 緊張を高める（A→B）0.70　緊張を下げる（B→A）1.30　比 1.86
b=-0.6: 緊張を高める（A→B）0.40　緊張を下げる（B→A）1.60　比 4.00
```
b=0 ならリーマン計量（行きと帰りは同じ）。|b| が大きいほど、行きと帰りの違いが大きい。

コードの全文

```
"""
敵の多様体 補助文書 03 コード 4：ランダース計量 ―― 行きと帰りで長さの違う定規
F(v) = |v| + b·v（|b| < 1）。背景が平らで b が一定なら、2 点を結ぶ最短の道は直線で、
A→B の長さは |AB| + b·(B−A)、B→A の長さは |AB| − b·(B−A) になる。
b を「緊張を高める方向」と逆向きに置くと、緊張を高める道は短く（易しく）、元に戻す道は長く（難しく）なる。数値は架空のもの。
"""
import numpy as np
A = np.array([0.0, 0.0]); B = np.array([1.0, 0.0])  # A：平時、B：緊張の高い状態
for bx in [0.0, -0.3, -0.6]:
    b = np.array([bx, 0.0])
    assert np.linalg.norm(b) < 1, "ランダース計量になるには |b| < 1 が必要"
    d_ab = np.linalg.norm(B - A) + b @ (B - A)
    d_ba = np.linalg.norm(A - B) + b @ (A - B)
    print(f"b={bx:+.1f}: 緊張を高める（A→B）{d_ab:.2f}  緊張を下げる（B→A）{d_ba:.2f}  比 {d_ba/d_ab:.2f}")
print("b=0 ならリーマン計量（行きと帰りは同じ）。|b| が大きいほど、行きと帰りの違いが大きい。")
```

第 6 章　政策の順序効果 ―― リー括弧と三重のリー括弧（tk05_bch_ordering.py）

本編の場面：第 2 章 2.7　**解説**：補助文書 02 第 7 章 7.3

実行結果

```
和だけ（順序を無視）：誤差 1.95e-01
リー括弧（2 次）まで：誤差 2.12e-02
三重のリー括弧（3 次）まで：誤差 3.47e-03
順序の入れ替え：exp(X)exp(Y)exp(Z) と exp(Z)exp(Y)exp(X) の差 3.62e-01
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 5：政策の順序の効果 ―― リー括弧と、三重のリー括弧（BCH の公式）
3 つの政策を、小さな行列 X, Y, Z の指数で表し、続けて実行した結果 exp(X)exp(Y)exp(Z) を、log をとった「まとめた効果」として計算する。
近似を、(1) 和だけ、(2) リー括弧（2 次）まで、(3) 三重のリー括弧（3 次）まで、と順に上げ、誤差を比べる。
```

```
"""
import numpy as np
from scipy.linalg import expm, logm
rng = np.random.default_rng(31)
s = 0.15
X, Y, Z = (s * rng.standard_normal((4, 4)) for _ in range(3))
br = lambda a, b: a @ b - b @ a
exact = logm(expm(X) @ expm(Y) @ expm(Z)).real
# BCH を２回使う：log(e^X e^Y) を３次まで、その結果と Z を、さらに３次まで
def bch3(A, B):
    return A + B + br(A, B) / 2 + (br(A, br(A, B)) + br(B, br(B, A))) / 12
o1 = X + Y + Z
W2 = X + Y + br(X, Y) / 2
o2 = W2 + Z + br(W2, Z) / 2
o3 = bch3(bch3(X, Y), Z)
for name, approx in [("和だけ（順序を無視）", o1), ("リー括弧（２次）まで", o2), ("三重のリー括弧（３次）まで", o3)]:
    print(f"{name}: 誤差 {np.linalg.norm(exact - approx):.2e}")
print("順序の入れ替え：exp(X)exp(Y)exp(Z) と exp(Z)exp(Y)exp(X) の差", f"{np.linalg.norm(expm(X)@expm(Y)@expm(Z) - expm(Z)@expm(Y)@expm(X)):.2e}")
```

第７章　命題 1.1 の条件２と条件３――検閲が近傍保存率と最小特異値に与える影響（tk06_manifold_conditions.py）

本編の場面：第２章 2.6　**解説**：補助文書 02 第４章 4.1・4.2

実行結果

```
正直なデータ（内在次元 35）：  35 番目（最小）の特異値  0.0335    26 番目  2.5496    27 番目  2.3064
検閲されたデータ（内在次元 26）：  35 番目（最小）の特異値  0.0093    26 番目  1.0541    27 番目  0.0108
→ 内在次元が 26 しかないと、27〜35 番目の特異値がほぼ 0 になり、35 次元の地図は 9 次元ぶん空回りする。
近傍保存率（正直なデータを 2 次元に圧縮）：  0.104
近傍保存率（ならされた検閲データを 2 次元に圧縮）：  0.904
→ 都合の悪い指標がならされると、近傍の構造が単純になり、保存率は『良く』見える。
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 6：命題 1.1 の条件 2 と条件 3 ―― 検閲が「近傍保存率」と「最小特異値」に与える影響
架空のデータ：35 個の指標のうち、本当に独立に動く方向（内在次元）は、正直なデータで 35、検閲されたデータで 26。
・条件 3：データを主成分（線形の近似）で 35 次元に写したとき、最小の特異値が 0 に近ければ、その方向は「空回り」している。
・条件 2：12 近傍の保存率。都合の悪い指標を「平穏な値」にならすと、近傍の構造が単純になり、保存率が上がる。
数値は架空のもの。本物の VAE（非線形）の代わりに、考え方を見るための線形の近似を使う。
"""
import numpy as np
rng = np.random.default_rng(35)
n, D = 1200, 35
def make(intrinsic):
    Z = rng.standard_normal((n, intrinsic))
    A = rng.standard_normal((intrinsic, D))
    return Z @ A + 0.01 * rng.standard_normal((n, D))
for name, k in [("正直なデータ（内在次元 35）", 35), ("検閲されたデータ（内在次元 26）", 26)]:
    X = make(k); X = X - X.mean(0)
    s = np.linalg.svd(X / np.sqrt(n), compute_uv=False)
    print(f"{name}: 35 番目（最小）の特異値 {s[-1]:.4f}  26 番目 {s[25]:.4f}  27 番目 {s[26]:.4f}")
print("→ 内在次元が 26 しかないと、27〜35 番目の特異値がほぼ 0 になり、35 次元の地図は 9 次元ぶん空回りする。")

def knn_preservation(X, Y, k=12):
    def nn(M):
        d = ((M[:, None, :] - M[None, :, :]) ** 2).sum(-1)
        np.fill_diagonal(d, np.inf)
        return np.argsort(d, axis=1)[:, :k]
    a, b = nn(X), nn(Y)
    return np.mean([len(set(a[i]) & set(b[i])) / k for i in range(len(X))])
# 現実のデータ：10 個の指標が、それぞれ独立に揺れる（地域ごとの不揃いな実態）
m = 400
honest = np.vstack([rng.standard_normal((m, 10)) * 0.8, rng.standard_normal((m, 10)) * 0.8 + 3.0])
# 検閲されたデータ：都合の悪い 8 つの指標（社会不安、隠れ債務など）が、上からの報告で「平穏な値」にならされる
```

```
censored = honest.copy(); censored[:, 2:] = censored[:, 2:].mean(0) + 0.05 * rng.standard_normal((2 * m, 8))
def pca2(X):
    Xc = X - X.mean(0); _, _, Vt = np.linalg.svd(Xc, full_matrices=False); return Xc @ Vt[:2].T
print(f"近傍保存率（正直なデータを 2 次元に圧縮）: {knn_preservation(honest, pca2(honest)):.3f}")
print(f"近傍保存率（ならされた検閲データを 2 次元に圧縮）: {knn_preservation(censored, pca2(censored)):.3f}")
print("→ 都合の悪い指標がならされると、近傍の構造が単純になり、保存率は『良く』見える。")
```

第 8 章　反復ベストレスポンスの収束と、ヘッセ行列による安定性の判定（tk07_iterated_best_response.py）

本編の場面：第 6 章 6.2・6.3　**解説**：補助文書 02 第 10 章 10.4・10.5

実行結果

```
縮小写像になる場合：合成写像のスペクトル半径 0.244 → 収束（反復 9 回）
縮小写像にならない場合：合成写像のスペクトル半径 1.328 → 収束せず（反復 500 回）
ヘッセ行列の固有値 [-2.032  0.832] → 鞍点型（正と負が混在）
→ 鞍点型の均衡は、ある方向の小さな揺れには戻るが、別の方向の揺れには崩れうる（本編の台湾海峡の均衡）。
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 7：反復ベストレスポンスと、均衡の安定性（ヘッセ行列の固有値）
2 人のプレイヤーの戦略 x, y（各 2 次元）。利得は 2 次関数で、相手の戦略に依存する。
最適応答 BR_x(y) = A y + a、BR_y(x) = B x + b。交互に応答を繰り返し、変化量が 1e-4 を下回ったら収束とみなす。
写像のスペクトル半径が 1 未満（縮小写像）なら収束し、1 を超えると発散・振動する。数値は架空のもの。
"""
import numpy as np
def run(A, B, a, b, tol=1e-4, maxit=500):
    x, y = np.zeros(2), np.zeros(2)
    for it in range(1, maxit + 1):
        x_new = A @ y + a
        y_new = B @ x_new + b
```

```
        step = np.linalg.norm(np.r_[x_new - x, y_new - y]); x, y = x_new, y_new
        if step < tol: return it, x, y, True
    return maxit, x, y, False
a, b = np.array([1.0, 0.5]), np.array([-0.5, 1.0])
for label, s in [("縮小写像になる場合", 0.6), ("縮小写像にならない場合", 1.4)]:
    A = s * np.array([[0.8, -0.3], [0.2, 0.7]]); B = s * np.array([[0.6, 0.4], [-0.5, 0.9]])
    rho = max(abs(np.linalg.eigvals(B @ A)))
    it, x, y, ok = run(A, B, a, b)
    print(f"{label}: 合成写像のスペクトル半径 {rho:.3f} → {'収束' if ok else '収束せず'}（反復 {it} 回）")
# 均衡点の安定性：利得関数のヘッセ行列の固有値
H = np.array([[-2.0, 0.3], [0.3, 0.8]])  # 均衡点での、ある政策の組の利得のヘッセ行列（架空）
ev = np.linalg.eigvalsh(H)
kind = "安定（すべて負）" if all(ev < 0) else ("鞍点型（正と負が混在）" if any(ev > 0) and any(ev < 0) else "不安定")
print(f"ヘッセ行列の固有値 {np.round(ev, 3)} → {kind}")
print("→ 鞍点型の均衡は、ある方向の小さな揺れには戻るが、別の方向の揺れには崩れうる（本編の台湾海峡の均衡）。")
```

第９章　相互情報量とデータ処理不等式、KL 情報量とワッサースタイン距離（tk08_information.py）

本編の場面：第３章 3.4、第１０章 10.2　**解説**：補助文書 02 第 15 章

実行結果

```
I(X;Y)  正直な観測 = 1.507 ビット
I(X;Y') 検閲した観測 = 0.605 ビット（増えることはない）
KL(現実 ‖ モデル) = inf（モデルが確率 0 とする場所に現実の質量があると発散する）
ワッサースタイン距離 = 0.604（遠くの質量も、動かす手間として有限に測れる）
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード８：相互情報量とデータ処理不等式、KL 情報量とワッサースタイン距離
(1) 現実 X を観測した Y から、都合の悪い値を「平穏」に書き換えた Y'（検閲）を作ると、I(X;Y') ≤ I(X;Y)。
(2) 現実の分布とモデルの分布が重ならない部分を持つと、KL 情報量は発散するが、ワッサースタイン距離は有限のまま。
数値は架空のもの。
```

```
"""
import numpy as np
from scipy.stats import wasserstein_distance
rng = np.random.default_rng(10)
def mutual_info(a, b):
    ja = np.unique(a); jb = np.unique(b); p = np.zeros((len(ja), len(jb)))
    for i, u in enumerate(ja):
        for j, v in enumerate(jb): p[i, j] = np.mean((a == u) & (b == v))
    pa, pb = p.sum(1, keepdims=True), p.sum(0, keepdims=True); nz = p > 0
    return float((p[nz] * np.log2(p[nz] / (pa @ pb)[nz])).sum())
X = rng.integers(0, 4, 20000)                  # 現実の状態（0=平穏 … 3=危機）
Y = np.where(rng.random(20000) < 0.9, X, rng.integers(0, 4, 20000))  # 正直な観測（1割は誤り）
Yc = np.where(Y >= 2, 0, Y)                    # 検閲：危うい値を「平穏」に書き換える
print(f"I(X;Y)  正直な観測 = {mutual_info(X, Y):.3f} ビット")
print(f"I(X;Y') 検閲した観測 = {mutual_info(X, Yc):.3f} ビット（増えることはない）")
# KL とワッサースタイン：モデル（検閲された分布）は、現実の遠くの裾（隠れ債務）に質量を持たない
bins = np.arange(0, 11)
real = np.r_[rng.normal(3, 1, 9000), rng.normal(9, 0.5, 1000)]   # 1割が遠くの裾
model = rng.normal(3, 1, 10000)
P, _ = np.histogram(real, bins=bins, density=True); Q, _ = np.histogram(model, bins=bins, density=True)
with np.errstate(divide="ignore", invalid="ignore"):
    kl = np.sum(np.where(P > 0, P * np.log(P / Q), 0.0))
print(f"KL(現実 ‖ モデル) = {kl}（モデルが確率 0 とする場所に現実の質量があると発散する）")
print(f"ワッサースタイン距離 = {wasserstein_distance(real, model):.3f}（遠くの質量も、動かす手間として有限に測れる）")
```

第 10 章　移転エントロピーと偶然の一致の検定（tk09_causal_tests.py）

本編の場面：第 5 章 5.3　**解説**：補助文書 02 第 14 章 14.3・14.4

実行結果

```
移転エントロピー TE(X→Y) = 0.2096 ビット    TE(Y→X) = 0.0007 ビット
重なりの回数 410、ずらした系列での平均 125.3、偶然でこれ以上重なる確率 p = 0.0010
```

→ 情報の流れの向きと、偶然では説明しにくいことは示せる。合意か便乗か（意図）は、データだけでは区別できない。

コードの全文

```
"""
敵の多様体 補助文書 03 コード 9：移転エントロピーと、偶然の一致の検定（循環ずらしの置換検定）
X（ロシアの関与が疑われる動き）が、1 期遅れて Y（中国の演習）に影響する架空の時系列を作る。
(1) 移転エントロピー TE(X→Y) と TE(Y→X) を、2 値化した系列で推定し、情報の流れの向きを比べる。
(2) Y を循環的にずらして（系列の性質を保ったまま）同時発生の数を数え直し、偶然でこれ以上重なる確率を求める。
数値は架空のもの。いずれも「意図」は示さない。
"""
import numpy as np
rng = np.random.default_rng(5)
T = 3000
x = (rng.random(T) < 0.2).astype(int)
y = np.zeros(T, int)
for t in range(1, T):
    p = 0.6 if x[t - 1] == 1 else 0.08
    y[t] = int(rng.random() < p)
def te(src, dst):
    # TE(src→dst) = Σ p(d1, d0, s0) log p(d1|d0,s0)/p(d1|d0)
    d1, d0, s0 = dst[1:], dst[:-1], src[:-1]; tot = 0.0
    for a in (0, 1):
        for b in (0, 1):
            for c in (0, 1):
                p_abc = np.mean((d1 == a) & (d0 == b) & (s0 == c))
                if p_abc == 0: continue
                p_a_bc = p_abc / np.mean((d0 == b) & (s0 == c))
                p_a_b = np.mean((d1 == a) & (d0 == b)) / np.mean(d0 == b)
                tot += p_abc * np.log2(p_a_bc / p_a_b)
    return tot
print(f"移転エントロピー TE(X→Y) = {te(x, y):.4f} ビット   TE(Y→X) = {te(y, x):.4f} ビット")
obs = np.sum(x[:-1] & y[1:])                  # X の直後に Y が起きた回数
null = [np.sum(x[:-1] & np.roll(y, s)[1:]) for s in range(50, T - 50, 3)]
p = (np.sum(np.array(null) >= obs) + 1) / (len(null) + 1)
print(f"重なりの回数 {obs}、ずらした系列での平均 {np.mean(null):.1f}、偶然でこれ以上重な
```

```
る確率 p = {p:.4f}")
print("→ 情報の流れの向きと、偶然では説明しにくいことは示せる。合意か便乗か（意図）は、データだけでは区別できない。")
```

第 11 章　シンプレクティック積分 —— 長期の計算でエネルギーが崩れないこと（tk10_symplectic.py）

本編の場面：第１２章　**解説**：補助文書 02 第 16 章

実行結果

```
オイラー法: 最初のエネルギー 0.500 → 20000 ステップ後 2.436e+21（比 4.871e+21）
ストルメール＝ヴェルレ法: 最初のエネルギー 0.500 → 20000 ステップ後 0.4998（比 0.9995）
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 10：シンプレクティック積分 —— 長期の計算で、エネルギーが崩れないこと
単振動 H = (p² + q²)/2 を、オイラー法とストルメール＝ヴェルレ法（シンプレクティック積分）で 20,000 ステップ計算し、
エネルギー H のずれを比べる。シンプレクティック積分が保証するのは「計算が崩れないこと」で、「予測が当たること」ではない。
"""
import numpy as np
h, N = 0.05, 20000
def euler(q, p):
    for _ in range(N): q, p = q + h * p, p - h * q
    return q, p
def verlet(q, p):
    for _ in range(N):
        p = p - 0.5 * h * q; q = q + h * p; p = p - 0.5 * h * q
    return q, p
H0 = 0.5
for name, f in [("オイラー法", euler), ("ストルメール＝ヴェルレ法", verlet)]:
    q, p = f(1.0, 0.0); H = 0.5 * (p * p + q * q)
    print(f"{name}: 最初のエネルギー {H0:.3f} → {N} ステップ後 {H:.4g}（比 {H/H0:.4g}）")
```

第 12 章　パーシステント・ホモロジーの β0 ―― 2 つのかたまりがつながる（tk11_beta0_bridge.py）

本編の場面：第 5 章 5.3　**解説**：補助文書 02 第 8 章 8.1

実行結果

```
r=0.3: 取っ手の前 β0=3　取っ手の後 β0=2
r=0.5: 取っ手の前 β0=2　取っ手の後 β0=1
r=0.8: 取っ手の前 β0=2　取っ手の後 β0=1
→ 取っ手が加わると、同じ半径で、2 つの塊が 1 つにつながる（β0 が 2 → 1）。
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 11：パーシステント・ホモロジーの β0 ―― 別々だった 2 つの塊がつながる
2 つの点の塊（中国の動き、ロシアの関与が疑われる動き）に、あとから両者をつなぐ点の列（取っ手）が加わる。
半径 r の球を各点に置いたとき、いくつのかたまりに分かれるか（β0）を、ユニオン・ファインドで数える。数値は架空のもの。
"""
import numpy as np
rng = np.random.default_rng(1)
A = rng.normal([0, 0], 0.4, (40, 2)); B = rng.normal([5, 0], 0.4, (40, 2))
handle = np.c_[np.linspace(0.8, 4.2, 12), 0.15 * rng.standard_normal(12)]
def beta0(P, r):
    n = len(P); parent = list(range(n))
    def find(i):
        while parent[i] != i: parent[i] = parent[parent[i]]; i = parent[i]
        return i
    d = np.sqrt(((P[:, None] - P[None]) ** 2).sum(-1))
    for i in range(n):
        for j in range(i + 1, n):
            if d[i, j] <= 2 * r: parent[find(i)] = find(j)
    return len({find(i) for i in range(n)})
for r in [0.3, 0.5, 0.8]:
    print(f"r={r}: 取っ手の前 β0={beta0(np.vstack([A, B]), r)}　取っ手の後 β0={beta0(np.vstack([A, B, handle]), r)}")
print("→ 取っ手が加わると、同じ半径で、2 つの塊が 1 つにつながる（β0 が 2 → 1）。")
```

第 13 章　ベイズ説得 ―― 嘘をつかずに判断を動かす（tk12_bayesian_persuasion.py）

本編の場面：第 9 章 9.6　**解説**：補助文書 02 第 13 章 13.4

実行結果

```
報告がなければ、中央は事前確率 0.6 で判断し、必ず介入する（介入の確率 1.00）
正直な報告（状態をそのまま伝える）なら、介入の確率 = 0.60
最適な見せ方：危機ありのとき確率 0.67、危機なしのとき確率 1.00 で『平穏』と報告
→ 介入の確率 = 0.20。『平穏』を聞いた中央の事後確率 = 0.499（閾値 0.5 をわずかに下回るように設計されている）
事後確率の平均 = 0.60（事前確率と同じ。信念は平均では偏らないが、行動は大きく動く）
```

コードの全文

```
"""
敵の多様体 補助文書 03 コード 12：ベイズ説得 ―― 嘘をつかずに、判断を動かす
ある省の状態は「危機あり」（事前確率 0.6）か「危機なし」。中央は、危機ありの事後確率が 0.5 以上なら介入する。
地方（送り手）は介入を避けたい。地方は報告の方針（各状態で「平穏」「警報」をどの確率で出すか）を先に決め、
中央はその方針を知ったうえで報告を読む。地方は嘘をつかない（方針は公開されている）。
数値は架空のもの。
"""
import numpy as np
prior, th = 0.6, 0.5
best = None
for a in np.linspace(0, 1, 201):        # 危機ありのとき「平穏」と報告する確率
    for b in np.linspace(0, 1, 201):    # 危機なしのとき「平穏」と報告する確率
        p_calm = prior * a + (1 - prior) * b; p_alarm = 1 - p_calm
        post_calm = prior * a / p_calm if p_calm > 0 else 0.0
        post_alarm = prior * (1 - a) / p_alarm if p_alarm > 0 else 0.0
        act = (p_calm if post_calm >= th else 0) + (p_alarm if post_alarm >= th else 0)
        if best is None or act < best[0] - 1e-12: best = (act, a, b, post_calm, post_alarm, p_calm)
act, a, b, pc, pa, p_calm = best
if pc > pa:  # 「平穏」を、介入につながらない側の報告として表示する
    a, b, pc, pa, p_calm = 1 - a, 1 - b, pa, pc, 1 - p_calm
print(f"報告がなければ、中央は事前確率 {prior} で判断し、必ず介入する（介入の確率 1.00）")
print(f"正直な報告（状態をそのまま伝える）なら、介入の確率 = {prior:.2f}")
print(f"最適な見せ方：危機ありのとき確率 {a:.2f}、危機なしのとき確率 {b:.2f} で『平穏』と報
```

告")
print(f"→ 介入の確率 = {act:.2f}。『平穏』を聞いた中央の事後確率 = {pc:.3f}（閾値 0.5 をわずかに下回るように設計されている）")
print(f"事後確率の平均 = {p_calm*pc + (1-p_calm)*pa:.2f}（事前確率と同じ。信念は平均では偏らないが、行動は大きく動く）")

第 14 章　N 人囚人のジレンマとメカニズムデザイン（tk13_n_person_dilemma.py）

本編の場面：第 9 章 9.1・9.2　**解説**：補助文書 02 第 13 章 13.1・13.2

実行結果

報奨 r=0.0: 隠す中で自分だけ正直にする利得の変化 -0.9965、正直な中で自分だけ隠す利得の変化 +0.9965 → 均衡：全員が隠す
報奨 r=1.5: 隠す中で自分だけ正直にする利得の変化 +0.5035、正直な中で自分だけ隠す利得の変化 -0.5035 → 均衡：全員が正直
社会的最適（全員が正直）の一人あたり利得 9.00、全員が隠すときの利得 0.00
→ 報奨が個人の損失を上回れば（誘因両立性）、正直に報告することが各自の最適な選択になる。

コードの全文

```
"""
敵の多様体 補助文書 03 コード 13：N 人囚人のジレンマと、メカニズムデザイン
2,850 人の幹部。正直に報告すると、全体の利益 B（天網の精度の向上）が人数に比例して増えるが、本人は損失 c（昇進の停止など）を負う。
各人の利得 = B × (正直な人の割合) − c × (自分が正直なら 1)。
制度（正直な報告への報奨 r、または損失の免除）を加えると、ナッシュ均衡が変わることを確かめる。数値は架空のもの。
"""

N, B, c = 2850, 10.0, 1.0
def payoff(honest_share, me_honest, reward=0.0):
    return B * honest_share - c * me_honest + reward * me_honest
for reward in [0.0, 1.5]:
    # 全員が隠しているとき、自分だけ正直にすると得か？（逸脱の誘因）
    gain_dev = payoff(1 / N, 1, reward) - payoff(0, 0, reward)
    # 全員が正直なとき、自分だけ隠すと得か？
    gain_hide = payoff((N - 1) / N, 0, reward) - payoff(1.0, 1, reward)
    eq = "全員が隠す" if gain_dev < 0 else ("全員が正直" if gain_hide < 0 else "混合")
    print(f"報奨 r={reward}: 隠す中で自分だけ正直にする利得の変化 {gain_dev:+.4f}、正直な中
```

```
で自分だけ隠す利得の変化 {gain_hide:+.4f} → 均衡：{eq}")
print(f"社会的最適（全員が正直）の一人あたり利得 {payoff(1.0, 1):.2f}、全員が隠すときの利得 {payoff(0, 0):.2f}")
print("→ 報奨が個人の損失を上回れば（誘因両立性）、正直に報告することが各自の最適な選択になる。")
```